segmentation fault on too many observables

https://github.com/quantumlib/qsim/issues/497

ROW 77

segmentation fault on too many observables #497

New issue

Closed #498

we-taper opened on Jan 12, 2022

Dear QSim developers, during a noisy VQE simulation on LiH, I notice that getting expectation value of a certain number of observables seems to result in a segmentation fault. Here's a simple demonstration:

```
import cirq
import qsimcirq

observables = [
    (0.023037232547556415)
    * cirq.X(cirq.GridQubit(0, 0))
    * cirq.X(cirq.GridQubit(1, 0))
    * cirq.X(cirq.GridQubit(2, 0))
    * cirq.Y(cirq.GridQubit(3, 0))
    * cirq.Z(cirq.GridQubit(4, 0))
    * cirq.Y(cirq.GridQubit(5, 0))
    * cirq.X(cirq.GridQubit(6, 0))
]
moments = [cirq.X.on(cirq.GridQubit(i, 0)) for i in range(12)]
circuit = cirq.Circuit(moments)

# ev_simulator = cirq.Simulator()
ev_simulator = qsimcirq.QSimSimulator(
    qsim_options={
        't': 2,
        # 'r': 100
    }
)
# Calculate expectation values for the given observables.
ev_results = ev_simulator.simulate_expectation_values(
    circuit,
    observables=observables,
)
ev_results = sum(ev_results)
print(ev_results)
```

A few notes that might be helpful:

- I have briefly tested the script above and found that whenever the number of observables is more than 7, the script will give segmentation fault.
- It has been tested on `qsimcirq==0.10.2` and the latest `qsimcirq== 0.11.1`.
- If switching to the TFQ (built from source with git tag `v0.5.1`) with its `Expectation` layer, the issue does not appear, which is a bit confusing since I thought TFQ uses QSim as its backend.

95-martin-orion on Jan 12, 2022 · edited by 95-martin-orion — Edits — Collaborator

This is known behavior, but we ought to have documentation for it (or guards against it) at the qsimcirq level as well. I'll take a look. (EDIT: correction - the code should route around this, but for some reason it doesn't.)

95-martin-orion self-assigned this on Jan 12, 2022

we-taper on Jan 13, 2022 — Author

Thanks for the quick reply. Is it because for over 6 qubits, the fused gate matrix becomes too large that it might be numerically efficient? Now the work around I think is to add pre-rotation gates and measure them in the standard basis (which I guess has been done in TFQ).

Assignees
95-martin-orion

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
Fix expectation values.
quantumlib/qsim

Notifications — Customize
Subscribe
You're not receiving notifications from this thread.

Participants

95-martin-orion self-assigned this on Jan 12, 2022

we-taper on Jan 13, 2022 — Author

Thanks for the quick reply. Is it because for over 6 qubits, the fused gate matrix becomes too large that it might be numerically efficient? Now the work around I think is to add pre-rotation gates and measure them in the standard basis (which I guess has been done in TFQ).

95-martin-orion on Jan 13, 2022 — Collaborator

> Is it because for over 6 qubits, the fused gate matrix becomes too large that it might be numerically efficient?

More or less, yes. The gate fusion code limits gates to six qubits due to larger gates being inefficient to simulate, and the expectation value code relies on the same behavior.

Looking into this, I found that there's a more expensive expectation value method that avoids this limit, and the pybind layer should automatically use it for more than six qubits:

qsim/pybind_interface/pybind_main.cpp
Lines 692 to 698 in 3d31b49

```
692     if (opsum_qubits <= 6) {
693       results.push_back(ExpectationValue<IO, Fuser>(opsum, simulator, state));
694     } else {
695       Fuser::Parameter param;
696       results.push_back(ExpectationValue<Fuser>(
697           param, opsum, state_space, simulator, state, scratch));
698     }
```

Clearly this switch is misbehaving somehow - if it wasn't, your code *should* work via the more expensive EV method.

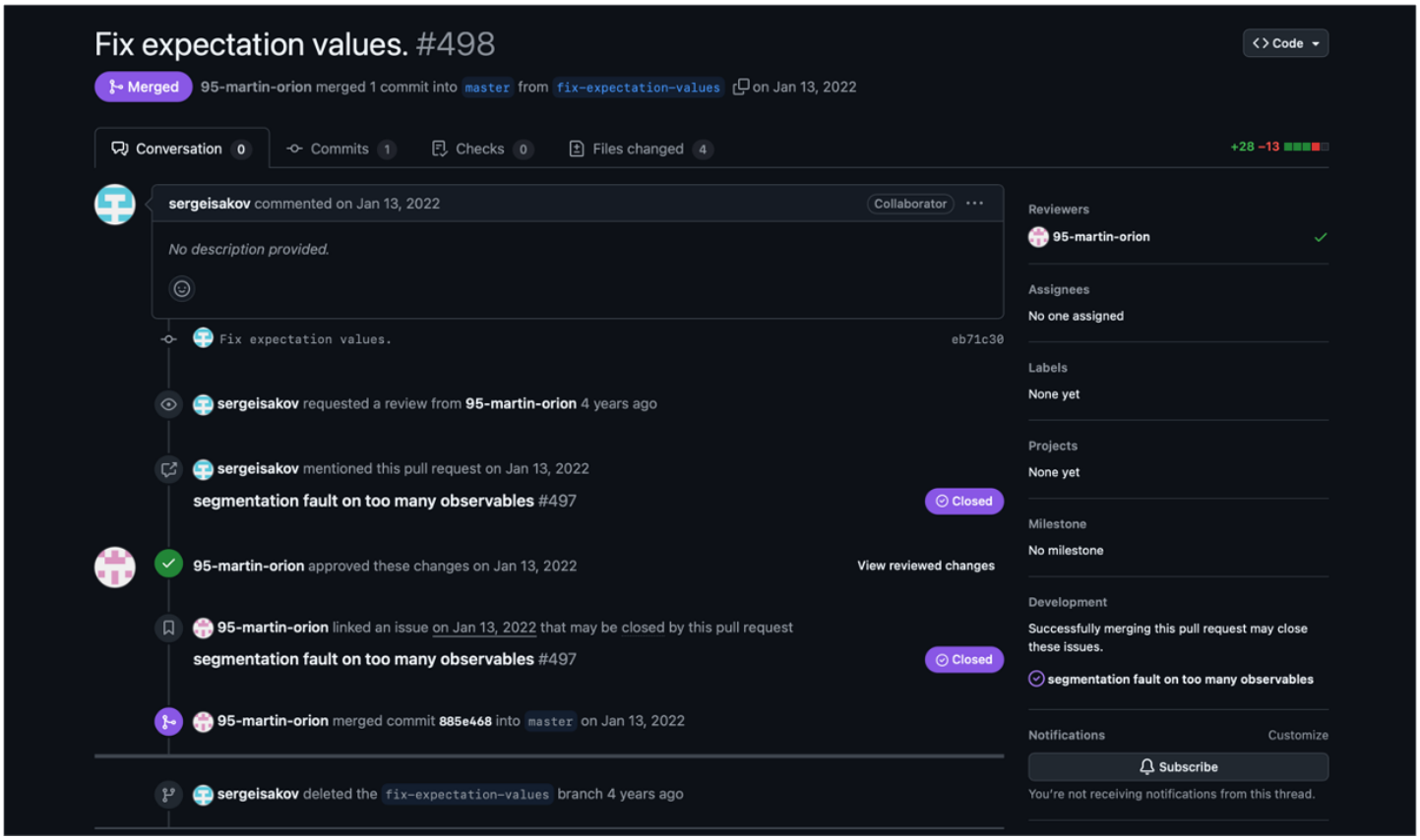
we-taper changed the title ~~segmentation fault on more than too many observables~~ segmentation fault on too many observables on Jan 13, 2022

sergeisakov on Jan 13, 2022 — Collaborator

The reason is that the more expensive method requires allocated scratch state. #498 should fix this issue.

95-martin-orion linked a pull request that will close this issue Fix expectation values. #498 on Jan 13, 2022

95-martin-orion closed this as completed in #498 on Jan 13, 2022


Fix expectation values. #498
Code
Merged
95-martin-orion merged 1 commit into master from fix-expectation-values on Jan 13, 2022
Conversation 0
Commits 1
Checks 0
Files changed 4
+28 −13
sergeisakov commented on Jan 13, 2022
Collaborator
No description provided.
Fix expectation values.
eb71c30
sergeisakov requested a review from 95-martin-orion 4 years ago
sergeisakov mentioned this pull request on Jan 13, 2022
segmentation fault on too many observables #497
Closed
95-martin-orion approved these changes on Jan 13, 2022
View reviewed changes
95-martin-orion linked an issue on Jan 13, 2022 that may be closed by this pull request
segmentation fault on too many observables #497
Closed
95-martin-orion merged commit 885e468 into master on Jan 13, 2022
sergeisakov deleted the fix-expectation-values branch 4 years ago
Reviewers
95-martin-orion
Assignees
No one assigned
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
segmentation fault on too many observables
Notifications
Customize
Subscribe
You're not receiving notifications from this thread.